

Plataforma web de jogos de geolocalização para treino em OSINT

GeoExplorer

RELATÓRIO INTERCALAR

Marcos Monteiro · 1902045
Orientador · Pedro Pestana

4 de maio de 2026

Índice

1.	<i>Capítulo 1 - Introdução</i>	5
1.1.	Enquadramento do projeto.....	5
1.2.	Necessidades identificadas	5
1.3.	Objetivos.....	6
1.4.	Âmbito e restrições	6
1.5.	Resultados esperados.....	7
1.6.	Calendário atualizado	7
1.7.	Organização do relatório	8
2.	<i>Capítulo 2 - Desenho</i>	9
2.1.	Tecnologias usadas	9
2.2.	Arquitetura do sistema	9
2.3.	Modelo de dados.....	10
2.4.	Algoritmos e lógica principal	11
2.5.	Interface do utilizador	12
2.6.	Decisões de arquitetura	13
2.7.	Riscos de desenho ainda ativos.....	13
3.	<i>Capítulo 3 - Implementação</i>	14
3.1.	Decomposição do trabalho e dependências.....	14
3.2.	Estado atual da implementação	14
3.3.	Validação técnica realizada.....	15
3.4.	Utilização de IA.....	15
3.5.	Plano para a fase restante	16
4.	<i>Capítulo 4 - Testes</i>	17
4.1.	Testes existentes	17
4.2.	Testes ainda a acrescentar	17
5.	<i>Capítulo 5 - Conclusões</i>	18
	<i>Bibliografia</i>	19
	<i>Anexo A - Destaques técnicos selecionados</i>	20

Lista de Figuras

Figura 1 - Diagrama C4 de contexto do GeoExplorer	10
Figura 2 - Diagrama C4 de contentores	10
Figura 3 - Modelo de dados relacional.....	11
Figura 4 - Ronda ativa e resultado com comparação cartográfica.....	12

Lista de Tabelas

Tabela 1 - Objetivos e estado à data do relatório intercalar	6
Tabela 2 - Calendário individual atualizado	7
Tabela 3 - Tecnologias utilizadas e justificação	9
Tabela 4 - Entidades principais da base de dados	11
Tabela 5 - Estado atual do conjunto de dados	11
Tabela 6 - Ecrãs principais da aplicação	12
Tabela 7 - Decisões de arquitetura existentes.....	13
Tabela 8 - Riscos ativos e mitigação.....	13
Tabela 9 - Decomposição do trabalho e dependências	14
Tabela 10 - Estado por componente	15
Tabela 11 - Evidências de validação intercalar	15
Tabela 12 - Plano de trabalho após o relatório intercalar	16
Tabela 13 - Cobertura automática atual	17

Resumo

O GeoExplorer é uma plataforma web de jogos de geolocalização pensada para treinar observação, análise visual e inferência geográfica em contexto de OSINT. O utilizador observa uma imagem real ou uma cena representativa, escolhe uma localização num mapa da Europa e recebe uma pontuação calculada a partir da distância ao local correto. Nesta fase intercalar, o projeto já deixou de ser apenas uma proposta. Comecei por estruturar a documentação, depois implementei uma primeira versão jogável do frontend, adicionei um backend em ASP.NET Core e, mais recentemente, passei a guardar sessões e rondas numa base de dados PostgreSQL.

O relatório intercalar segue o modelo disponibilizado para o relatório final, mas está preenchido de forma proporcional ao estado atual. Os Capítulos 1 e 2 estão escritos como base final da introdução e do desenho. O Capítulo 3 documenta o que já implementei, a dependência entre tarefas e o plano para a fase restante. Incluí ainda uma versão parcial dos capítulos de testes e conclusões, porque estes conteúdos já existem em parte e serão completados até à entrega final.

À data desta versão, o repositório já contém as principais peças da versão mínima viável (MVP). O frontend foi desenvolvido em React, TypeScript e Vite; o backend em ASP.NET Core .NET 8; e a execução local já está preparada com Docker Compose. Também já existe um conjunto de dados europeu com 158 locais reais, mapa com Leaflet/OpenStreetMap, gravação em PostgreSQL e testes automáticos. Na preparação deste relatório executei os testes existentes. Passaram 16 testes backend e 5 testes frontend. O build de produção do frontend também passou, embora com aviso normal de tamanho de bundle, que deixei registado como ponto de otimização futura.

Seguindo a orientação discutida na UC, não anexei o código completo ao relatório intercalar. Incluí apenas excertos técnicos curtos no anexo, escolhidos por mostrarem a ligação entre frontend, backend e regras de jogo. O código completo fica no repositório e será estabilizado até à entrega final.

1. Capítulo 1 - Introdução

Este capítulo enquadra o projeto, explicita o problema que motivou a sua escolha, delimita o âmbito do MVP e apresenta os objetivos, restrições e calendário atualizado. Procurei escrever esta parte como base do relatório final. O guia da unidade curricular é claro nesse ponto. O relatório intercalar deve ser o início do relatório final, não apenas um resumo de progresso.

1.1. ENQUADRAMENTO DO PROJETO

O GeoExplorer nasceu da ideia de construir uma aplicação web interativa para treino de geolocalização. A geolocalização a partir de imagens exige leitura de pistas visuais, comparação de padrões urbanos, atenção ao relevo, interpretação de sinais culturais e capacidade de fazer hipóteses com informação incompleta. Estas competências aparecem muitas vezes associadas a contextos de OSINT, investigação digital, jornalismo de verificação e análise de informação aberta.

Não tentei transformar o GeoExplorer numa alternativa completa a plataformas comerciais. Nesta fase, defini um MVP académico mais controlado: desafios na Europa, sessões individuais, mapa interativo, cálculo de pontuação e registo de resultados. Esta delimitação permite demonstrar o núcleo técnico sem depender de funcionalidades secundárias como autenticação, ranking global, multiplayer ou cobertura geográfica mundial.

A proposta aprovada descreve uma plataforma em que o utilizador observa uma localização e tenta identificar a posição aproximada num mapa. O sistema calcula a distância entre o palpite e a solução e transforma essa distância numa pontuação. Com base nessa proposta, comecei pela documentação de âmbito, requisitos, riscos e decisões de arquitetura. A seguir avancei para uma parte funcional de ponta a ponta, que já liga interface, regras de jogo, endpoints e base de dados.

1.2. NECESSIDADES IDENTIFICADAS

A principal necessidade é criar um ambiente simples e controlado para praticar inferência geográfica. Uma pessoa pode aprender a reconhecer pistas visuais, mas precisa de feedback rápido para perceber se a sua hipótese estava perto ou longe da realidade. O GeoExplorer responde a essa necessidade com uma interação curta, baseada em observar, marcar no mapa, submeter e comparar.

- Treinar observação visual aplicada a imagens de locais reais.
- Dar feedback quantitativo através de distância e pontuação.
- Manter a demonstração previsível, usando um conjunto de dados curado em vez de depender de chamadas externas no momento do jogo.
- Separar interface, backend e base de dados, mantendo a lógica de jogo isolada para a aplicação continuar evolutiva depois do MVP.

- Documentar o processo de decisão técnica de forma defensável para avaliação académica.

1.3. OBJETIVOS

O objetivo geral é desenvolver uma aplicação funcional para sessões de geolocalização na Europa. O utilizador deve conseguir iniciar uma sessão, jogar várias rondas, marcar palpites no mapa, receber pontuação e consultar os resultados da ronda e da sessão. Para tornar este objetivo verificável, mantive os critérios de aceitação definidos na proposta e fui acrescentando testes ou documentação à medida que cada parte ficava implementada.

Objetivo	Estado intercalar	Evidência no projeto
Iniciar sessão de jogo individual	Implementado em modo mock e suportado pelo backend	Frontend chama <code>createSession</code> ; o backend expõe <code>POST /api/sessions</code> .
Configurar número de rondas e temporizador	Implementado no frontend e validado no backend	<code>SetupPage</code> permite 3, 5 ou 7 rondas e modo livre/cronometrado; o backend valida 1 a 10 rondas e 15 a 180 segundos.
Apresentar desafios europeus	Implementado com conjunto de dados curado	<code>locations.json</code> contém 158 locais reais, 34 países e metadados visuais.
Submeter palpite geográfico	Implementado no mapa Leaflet e no endpoint	<code>EuropeGuessMap</code> recolhe coordenadas; endpoint <code>/guess</code> resolve a ronda.
Calcular distância e pontuação	Implementado	<code>GameSessionService</code> usa distância de Haversine e função exponencial de pontuação.
Registar resultados	Implementado para PostgreSQL quando as opções de base de dados estão ativas	Tabelas <code>game_sessions</code> e <code>session_rounds</code> , mais <code>GamePersistenceStore</code> .
Apresentar resultado final	Implementado no frontend	<code>SessionResultPage</code> mostra pontuação total, média, rondas e dispersão.
Preparar evolução futura	Parcialmente implementado	Alternância mock/backend, decisões de arquitetura e separação entre frontend, backend e base de dados.

Tabela 1 - Objetivos e estado à data do relatório intercalar

1.4. ÂMBITO E RESTRIÇÕES

O âmbito definido para o MVP é individual, europeu e sem autenticação. A aplicação não precisa, nesta versão, de ranking público, contas de utilizador, cobertura global ou multiplayer. Esta decisão é importante porque evita que o projeto se disperse antes de estar sólido no fluxo principal.

As restrições principais são tempo académico, previsibilidade da demonstração, licenciamento das imagens e integração entre tecnologias diferentes. A escolha de um conjunto de dados curado reduz a dependência de serviços externos durante a demo. A escolha de PostgreSQL em Docker reduz a dependência de fornecedor cloud nesta fase, mantendo uma porta aberta para Supabase ou outro alojamento futuro.

Nesta fase resisti a avançar para multiplayer, apesar de ser uma funcionalidade interessante. A prioridade passou a ser terminar bem a gravação em base de dados, estabilizar a integração entre frontend, backend e PostgreSQL, e deixar os critérios do MVP verificáveis.

1.5. RESULTADOS ESPERADOS

Na entrega final espero ter uma aplicação que possa ser executada localmente de forma reprodutível, demonstrada com dados reais e defendida a partir do relatório. O resultado esperado não é apenas uma interface bonita. É um sistema em que as decisões de arquitetura, o modelo de dados, os testes e as limitações estejam compreendidos.

- Sessão de jogo completa com várias rondas europeias.
- Modo livre e modo cronometrado estabilizados.
- Mapa real para seleção do palpite e comparação com a solução.
- Gravação de sessões, rondas, palpites, pontuação e resultados.
- Conjunto de dados expandido e validado, com origem, licença e atribuição visíveis.
- Testes automáticos associados aos critérios de aceitação mais críticos.
- Relatório final coerente com o código e com o histórico do repositório

1.6. CALENDÁRIO ATUALIZADO

O calendário abaixo atualiza o plano da proposta com o estado real observado até à semana 7. Mantive as datas formais da unidade curricular e ajustei a fase restante para refletir o trabalho que ainda falta estabilizar.

Semanas	Datas	Estado / trabalho previsto	Marco
1-2	17-28 mar	Definição do tema, proposta, sinopse, MVP, calendário e estrutura documental inicial.	Proposta
3-4	31 mar-11 abr	Requisitos MoSCoW, riscos, wireframes, C4, modelo de dados preliminar e estrutura do repositório.	Entrega 1
5	14-17 abr	Primeira versão jogável do frontend em React, TypeScript, Vite e Tailwind.	
6	20-25 abr	Arranque do backend ASP.NET Core, endpoints principais, Docker Compose e testes aos endpoints.	
7	28 abr-2 mai	Mapa real, 158 locais reais, metadados de fonte/licença, gravação em PostgreSQL e recuperação de sessões guardadas.	Demo interna
8	5-6 mai	Consolidar relatório intercalar, validar fluxo backend + PostgreSQL e corrigir falhas visíveis.	Intercalar
9-10	7-16 mai	Estabilizar UX, modo temporizado, consistência de resultados e integração ponta a ponta.	
11-12	19-30 mai	Expandir conjunto de dados para 250-300 locais, reforçar testes e consolidar execução Docker.	
13	2-6 jun	Melhorias não críticas, revisão de critérios de aceitação e avaliação realista do multiplayer.	
14	9-13 jun	Congelar funcionalidades, rever documentação e atualizar diagramas finais.	
15	16-20 jun	Preparar defesa, validar instalação e ensaiar demonstração.	Prep. defesa
16	24 jun	Submissão do relatório final e código associado.	Final

Tabela 2 - Calendário individual atualizado

1.7. ORGANIZAÇÃO DO RELATÓRIO

O Capítulo 2 apresenta o desenho da solução, incluindo tecnologias, arquitetura, modelo de dados, algoritmos principais, interface e decisões técnicas. O Capítulo 3 descreve o estado de implementação, as tarefas, as dependências e o plano restante. O Capítulo 4 fica já iniciado com os testes existentes e será expandido na entrega final com capturas de ecrã e testes de desempenho. O Capítulo 5 apresenta conclusões intercalares, dificuldades e trabalho futuro

2. Capítulo 2 - Desenho

Este capítulo descreve o desenho técnico do GeoExplorer à data do relatório intercalar. A arquitetura já não está apenas planeada. Várias decisões foram implementadas e testadas, embora ainda falte estabilizar a integração ponta a ponta com PostgreSQL ativo no fluxo completo do frontend.

2.1. TECNOLOGIAS USADAS

Área	Tecnologia	Justificação
Frontend	React 19 + TypeScript + Vite	Permite uma interface interativa, tipada e rápida de iterar, com ecrãs separados para início, configuração, ronda e resultados.
UI / Estilos	Tailwind CSS 4 + CSS próprio	Acelera a criação visual, mas manteve CSS específico para o ambiente de jogo e componentes reutilizáveis.
Mapa	Leaflet + tiles CARTO/OpenStreetMap	Permite recolher coordenadas reais no navegador sem implementar um motor cartográfico próprio.
Backend	ASP.NET Core .NET 8 Minimal API	Organiza a lógica de sessão, rondas, validação e resultados numa API simples e testável.
Base de dados	PostgreSQL + Entity Framework Core	PostgreSQL guarda os dados do jogo; Entity Framework Core é usado pelo backend para ler e escrever dados.
Execução	Docker Compose com perfis	Permite correr frontend em modo mock, frontend ligado ao backend e base de dados de forma mais reprodutível.
Testes	MSTest, WebApplicationFactory, Vitest	Cobre endpoints do backend, base de dados e alguma lógica de navegação/frontend.
Dados visuais	Wikimedia Commons	Fornece imagens reais com fonte, licença e atribuição documentadas no conjunto de dados.

Tabela 3 - Tecnologias utilizadas e justificação

2.2. ARQUITETURA DO SISTEMA

A arquitetura separa o sistema em frontend, backend e base de dados. O frontend é responsável pela experiência de jogo e pela recolha do palpite. O backend gere sessões, rondas, validação, pontuação e gravação de dados. A base de dados guarda catálogo de locais, sessões e rondas resolvidas.

Comecei com uma fonte de dados mock no frontend para validar rapidamente a experiência de jogo. Depois introduzi uma abstração entre dados mock e backend para não tornar o protótipo descartável. Esta decisão permitiu construir e demonstrar o frontend antes da ligação à base de dados estar completa, mantendo o caminho para os endpoints reais.

A abordagem Minimal API do ASP.NET Core também representou uma aprendizagem para mim. Onde trabalho estou mais próximo de código legacy e de abordagens mais tradicionais de backend. A parte mais exigente foi perceber como organizar bem os endpoints, a injeção de dependências e os testes HTTP sem tornar o backend difícil de manter.

GeoExplorer · C4 Contexto

Entrega 1 · visão de sistema, utilizador e dependências externas

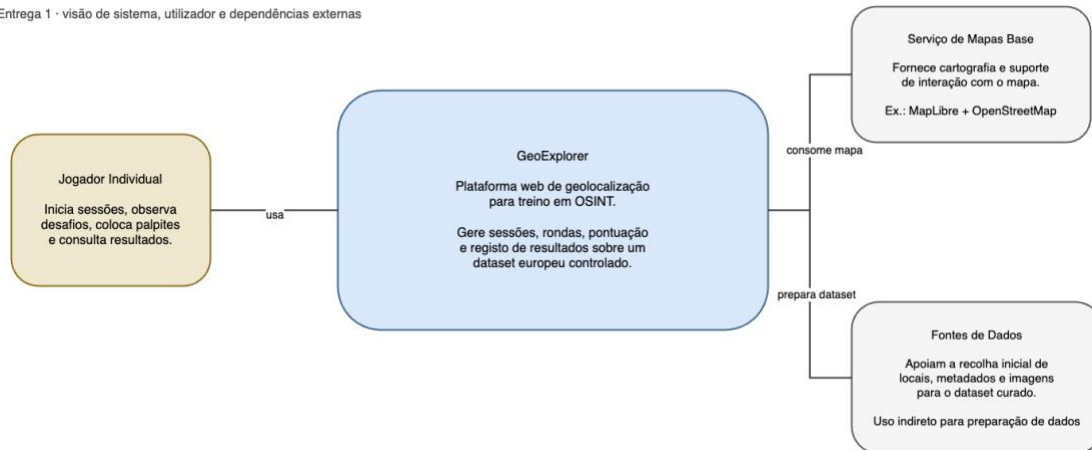


Figura 1 - Diagrama C4 de contexto do GeoExplorer

No nível de contexto, o jogador individual interage com o GeoExplorer através do navegador. O sistema depende de serviços de mapa base para visualização cartográfica e de fontes externas para preparação do conjunto de dados. Ainda assim, a execução do MVP não deve depender de recolha dinâmica desses dados a cada ronda.

GeoExplorer · C4 Contentores

Separação mínima entre interface, API e persistência

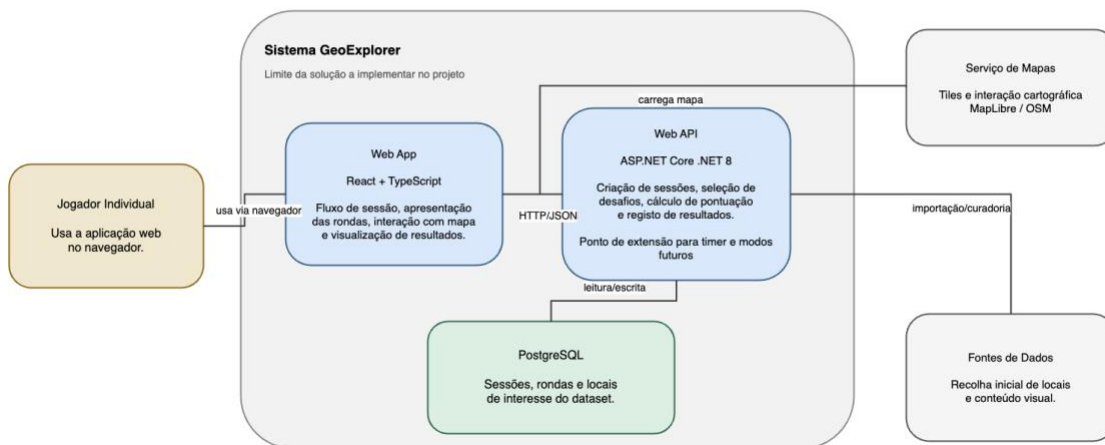


Figura 2 - Diagrama C4 de contentores

No nível de contentores, o frontend comunica com o backend por HTTP. O backend usa o catálogo de localizações e, quando as opções de PostgreSQL estão ativas, importa esse catálogo para a base de dados e guarda sessões/rondas. Esta separação ajuda a manter o frontend independente da origem dos dados e deixa espaço para futura autenticação ou multiplayer no backend.

2.3. MODELO DE DADOS

O modelo de dados foi desenhado para suportar primeiro o MVP individual. Existem três conceitos centrais, que são o catálogo de locais, a sessão de jogo e as rondas da sessão. O catálogo é relativamente estável; as sessões e rondas são dados produzidos durante a utilização.

GeoExplorer - Modelo de Dados

Esquema relacional mínimo para suportar o MVP

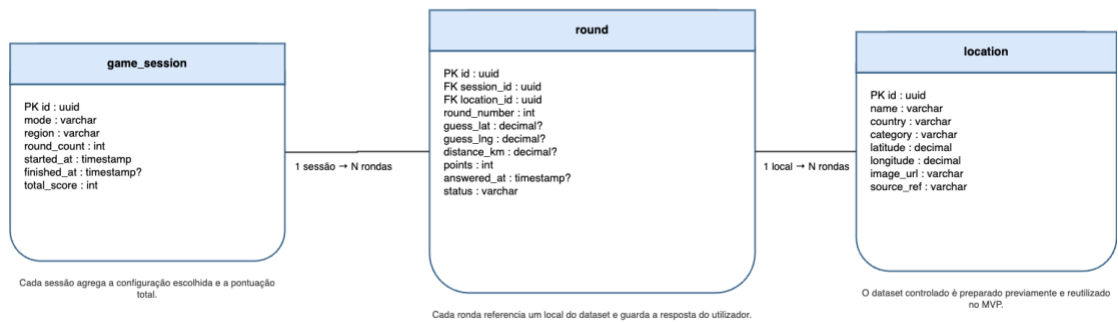


Figura 3 - Modelo de dados relacional

Entidade	Responsabilidade	Campos relevantes
locations	Guardar locais europeus disponíveis para rondas.	id, cidade, país, coordenadas, categoria, imagem, fonte, licença, pistas.
game_sessions	Guardar configuração e estado de cada sessão.	id, região, número de rondas, modo temporizado, pontuação total, estado, data de criação.
session_rounds	Guardar cada ronda de uma sessão.	session_id, location_id, número da ronda, palpite, distância, pontuação, motivo de resolução.

Tabela 4 - Entidades principais da base de dados

Métrica	Valor intercalar	Observação
Locais totais	158	Todos carregados a partir de src/database/seed/locations.json.
Países representados	34	Cobertura europeia suficiente para demonstração controlada.
Fonte visual	Wikimedia Commons	Todos os 158 locais têm metadados visuais reais.
Licenças principais	CC BY-SA 4.0, CC BY-SA 3.0, CC0	A licença e a atribuição são apresentadas nos resultados.
Categorias	historic-core, landmark, plaza, waterfront, fortress, canal-city, entre outras	As categorias suportam variedade visual sem alterar a lógica do jogo.

Tabela 5 - Estado atual do conjunto de dados

2.4. ALGORITMOS E LÓGICA PRINCIPAL

A lógica de jogo passa por quatro passos. O backend seleciona locais aleatórios sem repetição dentro da sessão. Cada ronda é construída sem expor as coordenadas corretas. Depois de o utilizador submeter o palpite, o backend calcula a distância, atribui a pontuação e avança o estado da sessão.

A distância é calculada com a fórmula de Haversine, usando raio médio da Terra de 6371 km. A pontuação atual usa uma função exponencial, $5000 * \exp(-\text{distância} / 650)$, arredondada e limitada a zero. Esta regra dá pontuação alta a palpites próximos e degrada progressivamente quando a distância aumenta.

A regra de pontuação ainda pode ser calibrada depois de mais sessões de teste. Nesta fase escolhi uma função simples, determinística e fácil de explicar em defesa.

2.5. INTERFACE DO UTILIZADOR

A interface foi desenhada como um fluxo de jogo curto. O utilizador começa na página inicial, configura a sessão, joga cada ronda e consulta o resultado antes de avançar. No fim, vê um resumo com a pontuação e a dispersão das rondas. Os wireframes iniciais estão em docs/design/wireframes.pdf e docs/design/Wireframe.pdf; o frontend atual já ultrapassa esses esboços ao incluir mapa real e imagens reais.

Ecrã	Objetivo	Estado
Início	Apresentar o jogo e permitir iniciar sessão ou tutorial.	Implementado.
Configuração	Escolher rondas e ritmo livre/cronometrado.	Implementado.
Ronda	Mostrar imagem/cena, pistas e mapa para palpite.	Implementado com mapa real e minimapa expansível.
Resultado da ronda	Mostrar local correto, palpite, distância, pontuação, fonte e licença.	Implementado.
Resultado da sessão	Consolidar pontuação, média, rondas e dispersão geográfica.	Implementado.
Tutorial	Explicar o fluxo ao primeiro uso.	Implementado com progresso guardado em localStorage.

Tabela 6 - Ecrãs principais da aplicação

A figura seguinte mostra duas superfícies já implementadas: a ronda ativa com mapa real aberto e o resultado da ronda com comparação cartográfica. Incluí esta evidência visual porque ajuda a relacionar o desenho descrito com o estado concreto da aplicação.

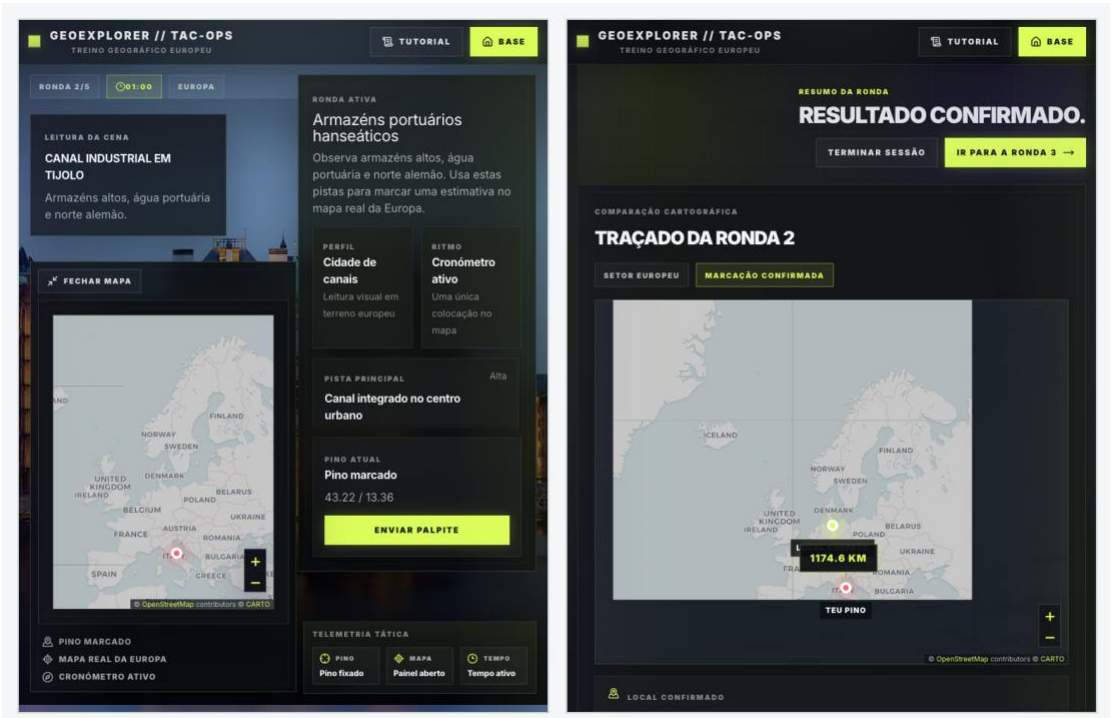


Figura 4 - Ronda ativa e resultado com comparação cartográfica

2.6. DECISÕES DE ARQUITETURA

As decisões principais estão documentadas em ficheiros próprios de arquitetura. Isto ajuda a não transformar escolhas técnicas em decisões implícitas. Também permite explicar no relatório porque rejeitei alternativas como Angular, frontend sem framework, Supabase completo nesta fase ou Docker apenas no fim.

Ficheiro	Decisão	Alternativas consideradas	Estado
ADR-001	Frontend em React + TypeScript + Vite	Angular; HTML/CSS/JS sem framework	Aceite
ADR-002	Backend em ASP.NET Core .NET 8 e PostgreSQL	FastAPI; ficheiros locais; Supabase completo; Turso/libSQL	Aceite, atualizado em 30 abr
ADR-003	Abstração mock/backend e Docker Compose com perfis	Mock sem abstração; Docker apenas no fim	Aceite

Tabela 7 - Decisões de arquitetura existentes

2.7. RISCOS DE DESENHO AINDA ATIVOS

Os riscos ainda mais relevantes são a integração completa entre frontend, backend e PostgreSQL, a expansão do conjunto de dados sem perder qualidade de licenciamento, e o tempo necessário para testes e relatório final. Também deixei registado que o multiplayer só deve avançar se o núcleo estiver estável, porque introduz salas, timers, reconexões e sincronização de estado.

Risco	Impacto	Mitigação
Divergência entre mock e backend	Médio	Modelos de dados alinhados e testes de fluxo HTTP.
Conjunto de dados inconsistente ou com licenças incompletas	Alto	Testes de validação e metadados obrigatórios.
Integração tardia com PostgreSQL	Médio	Parte funcional de ponta a ponta e opções UsePostgresCatalog/UsePostgresPersistence.
Deriva de âmbito para multiplayer	Alto	Manter multiplayer como funcionalidade futura, dependente do núcleo.
Testes insuficientes	Médio	Expandir cobertura a fluxos frontend e casos-limite do backend.

Tabela 8 - Riscos ativos e mitigação

3. Capítulo 3 – Implementação

Este capítulo documenta o estado atual de implementação no momento do relatório intercalar. Como o projeto é individual, todas as tarefas foram executadas por mim. A decomposição abaixo mostra também a dependência entre tarefas, ou seja, aquilo que tinha de estar suficientemente feito antes de eu avançar para a tarefa seguinte.

3.1. DECOMPOSIÇÃO DO TRABALHO E DEPENDÊNCIAS

Nesta tabela, a coluna "Depende de" substitui códigos técnicos de tarefas por uma descrição direta. Quando aparece "Nenhuma", significa que a tarefa podia começar logo no início. Quando aparece outra tarefa, significa que essa tarefa anterior tinha de estar pronta ou estável para a seguinte fazer sentido.

N.º	Tarefa	Depende de	Estado
1	Definir tema, sinopse, MVP e calendário	Nenhuma	Concluído
2	Documentar requisitos MoSCoW, riscos e changelog	Tema e MVP definidos	Em atualização contínua
3	Criar wireframes e artefactos de arquitetura	Tema, âmbito e requisitos iniciais	Concluído para o relatório intercalar
4	Configurar frontend React/Vite/TypeScript	Tema e fluxo principal definidos	Concluído
5	Implementar fluxo mock jogável	Frontend base configurado	Concluído
6	Adicionar mapa real e recolha de coordenadas	Fluxo jogável inicial	Concluído
7	Criar backend e endpoints de comunicação	MVP definido e fluxo frontend inicial	Concluído
8	Adicionar Docker Compose com perfis	Frontend e backend criados	Concluído
9	Curar conjunto de dados real europeu	Fluxo de rondas definido	Parcial, com 158 de 250-300 locais previstos
10	Guardar catálogo, sessões e rondas em PostgreSQL	Backend e Docker Compose	Implementado; falta validação completa no fluxo frontend + backend
11	Reforçar testes frontend/backend	Fluxo jogável, backend e base de dados inicial	Parcial
12	Preparar relatório e defesa	Trabalho técnico e documentação de suporte	Em curso

Tabela 9 - Decomposição do trabalho e dependências

3.2. ESTADO ATUAL DA IMPLEMENTAÇÃO

O frontend já permite executar uma sessão em modo mock, com ecrãs de início, configuração, ronda, resultado da ronda, resultado final e tutorial. A ronda usa uma composição visual com imagem real quando disponível e um mapa Leaflet para o palpite. Também implementei uma fonte de dados configurável, que permite trocar entre mock e backend.

O backend expõe endpoints para criar sessão, obter ronda atual, submeter palpite, resolver timeout e consultar resultados. A lógica de sessão impede múltiplas submissões na mesma ronda e valida limites como região suportada, número de rondas e duração do temporizador.

A base de dados PostgreSQL já guarda sessões e rondas quando está ativa. Também implementei recuperação de sessões guardadas. Se uma sessão não estiver em memória, o backend tenta reconstruí-la a partir da base de dados. Acrescentei ainda um endpoint de diagnóstico simples para contar leituras e escritas na base de dados, porque quero observar o padrão real antes de decidir qualquer alojamento gerido.

Componente	Implementado	Pendente
Frontend	Fluxo jogável, mapa real, resultados, tutorial e alternância mock/backend.	Validar sessão completa ligada ao backend com PostgreSQL ativo; melhorar testes de componentes.
Backend	Minimal API, endpoints, validação, pontuação, timeout, resultados.	Rever casos-limite e preparar endpoints adicionais só se forem necessários ao MVP.
Base de dados	Schema SQL, catálogo, sessões, rondas, recuperação e métricas.	Validar com Docker Compose durante sessões completas e rever migrações finais.
Conjunto de dados	158 locais reais com Wikimedia Commons, licença, atribuição e data de verificação.	Expandir para 250-300 locais e manter validação automática.
Testes	16 testes backend, 5 frontend, build de produção executado.	Cobrir mais fluxo visual e integração ponta a ponta.
Documentação	README, requisitos, riscos, changelog, decisões de arquitetura e relatório intercalar.	Completar capítulos 4 e 5 para relatório final e normalizar nomes canónicos dos diagramas.

Tabela 10 - Estado por componente

3.3. VALIDAÇÃO TÉCNICA REALIZADA

Durante a preparação deste relatório executei os testes existentes para não documentar um estado não verificado. O backend passou com 16 testes; o frontend passou com 5 testes Vitest. Também executei o build do frontend, que passou e gerou os artefactos de produção.

Comando	Resultado
dotnet test GeoExplorerer.slnx	Passed: 16; Failed: 0; Skipped: 0; duração aproximada 523 ms nos testes backend.
npm test -- --run	2 ficheiros de teste, 5 testes passados no frontend.
npm run build	Build Vite concluído; aviso de chunk superior a 500 kB para análise futura.

Tabela 11 - Evidências de validação intercalar

3.4. UTILIZAÇÃO DE IA

Usei ferramentas de IA como apoio ao planeamento, estruturação de documentação, discussão de arquitetura e criação de testes iniciais. Também usei GitHub Copilot para sugestões pontuais durante desenvolvimento. Procurei manter esta utilização como apoio e não como substituição da compreensão técnica. As decisões documentadas nos ficheiros de arquitetura são as decisões que pretendo conseguir defender.

No relatório final vou manter uma secção de referências sobre IA utilizada. O guia da unidade curricular considera aceitável o uso de IA desde que seja referenciado e que o estudante compreenda o que entrega.

3.5. PLANO PARA A FASE RESTANTE

Período	Trabalho principal	Resultado esperado
5-9 mai	Validar frontend ligado ao backend com PostgreSQL ativo.	Fluxo ponta a ponta demonstrável sem depender apenas do mock.
12-16 mai	Ajustar UX, temporizador, resultados e mensagens de erro.	Experiência de jogo mais estável e compreensível.
19-30 mai	Expandir conjunto de dados e reforçar testes.	250-300 locais e cobertura mínima dos critérios de aceitação.
2-6 jun	Rever limitações, dívida técnica e eventual extensão controlada.	Decisão consciente sobre multiplayer ou adiamento.
9-20 jun	Congelar funcionalidades, completar relatório final e preparar defesa.	Versão candidata de entrega e demonstração ensaiada.

Tabela 12 - Plano de trabalho após o relatório intercalar

4. Capítulo 4 - Testes

Este capítulo ainda não está completo. Nesta fase, deixei-o iniciado com os testes já existentes, para servir de base ao relatório final.

4.1. TESTES EXISTENTES

Área	Testes existentes	O que verificam
Conjunto de dados	SeedLocationDatasetTests	IDs únicos, coordenadas dentro da Europa suportada e metadados obrigatórios de media real.
Serviço de jogo	GameSessionServiceTests	scenelimage, metadados visuais, seleção única de locais, resultado com media e schema base.
API	GameApiIntegrationTests	health, diagnóstico, fluxo HTTP completo e erro para região não suportada.
Base de dados	GamePersistenceStoreTests	criação, resolução, conclusão, recuperação e métricas de leituras/escritas.
Frontend	navigation.test.ts e AppSidebar.test.tsx	decisão da fase de análise e comportamento do botão de análise.

Tabela 13 - Cobertura automática atual

4.2. TESTES AINDA A ACRESCENTAR

- Testes de integração com frontend ligado ao backend real numa sessão completa.
- Testes de componentes para SetupPage, RoundPage e resultados.
- Testes de casos-limite do temporizador, incluindo timeout com e sem palpite.
- Testes de base de dados com PostgreSQL real em Docker, para complementar InMemoryDatabase.
- Medição simples de desempenho, incluindo tempo de criação de sessão, carregamento de ronda e resposta da API.
- Capturas de ecrã finais relacionadas diretamente com os critérios de aceitação.

5. Capítulo 5 - Conclusões

A meio do projeto, considero que a decisão mais importante foi manter o âmbito controlado. No início considerei acrescentar mais modos e funcionalidades. Com o desenvolvimento, ficou claro que o valor principal está em fechar bem o ciclo base: observar, marcar, pontuar e guardar resultados. O projeto já tem uma base jogável e uma arquitetura coerente, mas ainda precisa de validação ponta a ponta com a base de dados ativa.

As principais dificuldades foram a articulação entre frontend, backend e dados reais, e a necessidade de tratar licenças de imagem com rigor. A introdução do mapa real também mudou a perceção da interface. Deixou de bastar um minimapa mock, porque a precisão do palpite depende de uma interação cartográfica utilizável.

Para a fase final, vou concentrar o esforço em estabilização, testes e documentação. Se houver tempo, poderei preparar uma extensão pequena, mas só depois de confirmar que os critérios de aceitação do MVP estão cobertos. O relatório final deverá mostrar não apenas que a aplicação funciona, mas também porque foi construída desta forma e quais foram as limitações assumidas

Bibliografia

Agile Business Consortium. (s.d.). MoSCoW prioritisation. Consultado em 2 de maio de 2026, em <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>

Brown, S. (s.d.). The C4 model for visualising software architecture. Consultado em 2 de maio de 2026, em <https://c4model.com/>

Docker Inc. (s.d.). Docker Compose overview. Consultado em 2 de maio de 2026, em <https://docs.docker.com/compose/>

Leaflet. (s.d.). Leaflet documentation. Consultado em 2 de maio de 2026, em <https://leafletjs.com/reference.html>

Microsoft. (s.d.). ASP.NET Core documentation. Consultado em 2 de maio de 2026, em <https://learn.microsoft.com/aspnet/core/>

Microsoft. (s.d.). Entity Framework Core documentation. Consultado em 2 de maio de 2026, em <https://learn.microsoft.com/ef/core/>

OpenStreetMap contributors. (s.d.). Copyright and license. Consultado em 2 de maio de 2026, em <https://www.openstreetmap.org/copyright>

PostgreSQL Global Development Group. (s.d.). PostgreSQL documentation. Consultado em 2 de maio de 2026, em <https://www.postgresql.org/docs/>

React. (s.d.). React documentation. Consultado em 2 de maio de 2026, em <https://react.dev/>

Vite. (s.d.). Vite guide. Consultado em 2 de maio de 2026, em <https://vite.dev/guide/>

Wikimedia Commons. (s.d.). Reusing content outside Wikimedia. Consultado em 2 de maio de 2026, em https://commons.wikimedia.org/wiki/Commons:Reusing_content_outside_Wikimedia

Anexo A - Destaques técnicos selecionados

Nesta fase optei por não reproduzir grandes blocos de código. Os excertos seguintes funcionam apenas como evidência curta das partes mais representativas do MVP atual. O primeiro mostra os endpoints usados pelo frontend para criar sessões, consultar rondas e submeter palpites. O segundo mostra como uma ronda é resolvida e como a distância é transformada em pontuação. Incluí também o ponto em que o mapa transforma um clique em coordenadas e um excerto resumido do schema SQL, porque estes elementos mostram o caminho principal do jogo sem anexar o projeto inteiro.

Código A.1 - Endpoints principais de sessão e ronda

```
35 api.MapPost("/sessions", (CreateSessionRequest request, GameSessionService gameSessions) =>
36 {
37     try
38     {
39         return Results.Ok(gameSessions.CreateSession(request));
40     }
41     catch (GameFlowException exception)
42     {
43         return Results.Problem(exception.Message, statusCode: exception.StatusCode);
44     }
45 });
46
47 api.MapGet("/sessions/{sessionId}/current-round", (string sessionId, GameSessionService
gameSessions) =>
48 {
49     try
50     {
51         return Results.Ok(gameSessions.GetCurrentRound(sessionId));
52     }
53     catch (GameFlowException exception)
54     {
55         return Results.Problem(exception.Message, statusCode: exception.StatusCode);
56     }
57 });
58
59 api.MapPost("/sessions/{sessionId}/rounds/{roundId}/guess", (
60     string sessionId,
61     string roundId,
62     GuessRequest request,
63     GameSessionService gameSessions) =>
64 {
65     if (request.Guess is null)
66     {
67         return Results.Problem("É necessário fornecer um palpite válido.", statusCode:
StatusCodes.Status400BadRequest);
68     }
```

```

69
70     try
71     {
72         return Results.Ok(gameSessions.SubmitGuess(sessionId, roundId, request.Guess));
73     }
74     catch (GameFlowException exception)
75     {
76         return Results.Problem(exception.Message, statusCode: exception.StatusCode);
77     }
78 });

```

Código A.2 - Resolução de ronda e pontuação

```

197     private RoundResolutionResponse ResolveRound(
198         SessionState session,
199         RoundState round,
200         GuessCoordinatesDto? guess,
201         string resolution)
202     {
203         var boundedGuess = guess is null ? null : ClampGuess(guess);
204         double? distanceKm = boundedGuess is null
205             ? null
206             : HaversineDistanceKm(
207                 boundedGuess.Latitude,
208                 boundedGuess.Longitude,
209                 round.Location.Latitude,
210                 round.Location.Longitude);
211
212         var result = new RoundResultDto(
213             round.Id,
214             round.RoundNumber,
215             round.Location.Title,
216             round.Location.City,
217             round.Location.Country,
218             round.Location.Latitude,
219             round.Location.Longitude,
220             boundedGuess,
221             distanceKm is null ? 0 : ScoreFromDistance(distanceKm.Value),
222             distanceKm,
223             resolution,
224             session.Config.Timed,
225             BuildMedia(round.Location.Media),
226             round.Location.Clues
227             .Select(clue => new ChallengeClueDto(clue.Label, clue.Value,
228                 clue.Confidence))
229             .ToList());
230
231         round.Result = result;
232         session.CurrentRoundIndex += 1;

```

```
232 PersistRoundResolution(session, round, result);
```

Código A.3 - Distância de Haversine e fórmula de pontuação

```
297 private static double HaversineDistanceKm(double latitudeA, double longitudeA, double
latitudeB, double longitudeB)
298 {
299     const double earthRadiusKm = 6371;
300     static double ToRadians(double value) => value * Math.PI / 180;
301
302     var deltaLat = ToRadians(latitudeB - latitudeA);
303     var deltaLng = ToRadians(longitudeB - longitudeA);
304     var latA = ToRadians(latitudeA);
305     var latB = ToRadians(latitudeB);
306     var a = Math.Pow(Math.Sin(deltaLat / 2), 2) +
307           Math.Cos(latA) * Math.Cos(latB) * Math.Pow(Math.Sin(deltaLng / 2), 2);
308
309     return 2 * earthRadiusKm * Math.Asin(Math.Sqrt(a));
310 }
311
312 private static int ScoreFromDistance(double distanceKm)
313 {
314     return Math.Max(0, (int)Math.Round(5000 * Math.Exp(-distanceKm / 650)));
315 }
```

Código A.4 - Clique no mapa e coordenadas do palpite

```
172 useEffect(() => {
173     const map = mapRef.current;
174     if (!map) {
175         return;
176     }
177
178     const handleClick = (event: L.LeafletMouseEvent) => {
179         if (!canInteract || !EUROPE_BOUNDS.contains(event.latlng)) {
180             return;
181         }
182
183         const latitude = Number(event.latlng.lat.toFixed(4));
184         const longitude = Number(event.latlng.lng.toFixed(4));
185
186         onGuessChange?.({
187             latitude,
188             longitude,
189             label: buildGuessLabel(latitude, longitude),
190         });
191     };
```

```

192
193     map.on("click", handleClick);
194     return () => {
195         map.off("click", handleClick);
196     };
197 }, [canInteract, onGuessChange]);

```

Código A.5 - Schema resumido de locais e sessões

```

1 CREATE EXTENSION IF NOT EXISTS pgcrypto;
2
3 CREATE TABLE IF NOT EXISTS locations (
4     id TEXT PRIMARY KEY,
5     title TEXT NOT NULL,
6     city TEXT NOT NULL,
7     country TEXT NOT NULL,
8     region TEXT NOT NULL,
9     category TEXT NOT NULL,
10    latitude DOUBLE PRECISION NOT NULL,
11    longitude DOUBLE PRECISION NOT NULL,
12    scene_label TEXT NOT NULL,
13    scene_note TEXT NOT NULL,
14    scene_image TEXT NULL,
15    media_source_provider TEXT NULL,
16    image_url TEXT NULL,
17    image_source_url TEXT NULL,
18    image_attribution TEXT NULL,
19    image_license TEXT NULL,
20    image_license_url TEXT NULL,
21    street_view_provider TEXT NULL,
22    street_view_url TEXT NULL,
23    media_verified_at DATE NULL,
24    prompt TEXT NOT NULL,
25    visual_gradient JSONB NOT NULL,
26    clues JSONB NOT NULL
27 );
28
29 CREATE TABLE IF NOT EXISTS game_sessions (
30     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
31     region TEXT NOT NULL,
32     round_count INTEGER NOT NULL,
33     timed BOOLEAN NOT NULL,
34     round_time_seconds INTEGER NULL,
35     total_score INTEGER NOT NULL DEFAULT 0,
36     status TEXT NOT NULL DEFAULT 'created',
37     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
38 );

```


39

```
40 CREATE TABLE IF NOT EXISTS session_rounds (  
41     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
42     session_id UUID NOT NULL REFERENCES game_sessions(id) ON DELETE CASCADE,  
43     location_id TEXT NOT NULL REFERENCES locations(id),  
44     round_number INTEGER NOT NULL,  
45     status TEXT NOT NULL DEFAULT 'pending',
```

Área	Ficheiro ou pasta	Finalidade
Backend	Program.cs	Endpoints usados pelo frontend para sessões, rondas e palpites.
Regras de jogo	GameSessionService.cs	Seleção de rondas, validação, distância e pontuação.
Frontend	EuropeGuessMap.tsx	Mapa Leaflet e recolha de coordenadas do palpite.
Base de dados	001-init.sql	Estrutura inicial de locais, sessões e rondas.
Testes	backend.Tests e testes frontend	Validação automática do estado descrito no relatório.

Tabela A.1 - Evidência complementar no repositório